FasoFree

Réseau de Livraison Multi-Niveaux

Plan de développement — présentation aux collaborateurs

Version 2.0 · Octobre 2026

Statut : proposition — en attente de validation

1. Contexte et problème

FasoFree garantit déjà une expérience de commande complète (marchands, livreurs, clients, paiements). Le point qui bloque la croissance est la disponibilité des livreurs : le système sait très bien distribuer une course quand un livreur est disponible, mais il ne sait pas garantir la livraison quand aucun livreur n'est disponible.

Aujourd'hui, « aucun livreur » est traité comme une fin de course (simple alerte au marchand). C'est le trou fonctionnel principal : aucune solution de secours n'existe.

- Volume insuffisant de livreurs actifs connectés et à proximité (≤ 10 km).
- Aucun débordement vers des flottes partenaires ou des agences.
- Aucun suivi de délai (SLA) opposable sur le cycle de prise en charge.

2. Ce que nous avons déjà (acquis à préserver)

Le socle actuel est solide et ne doit pas être réécrit — il sera enrichi par une couche supérieure :

Brique	Fonction	Niveau
Dispatch automatique	Scoring des livreurs (distance 60 % + note 40 %, contraintes vélo, ≤ 10 km) puis notification des 3 meilleurs candidats	🟢 8,5/10
Anti-course-pillée	Transaction SERIALIZABLE + verrou pessimiste sur l'acceptation : deux livreurs ne peuvent pas prendre la même commande	🟢 9,5/10
Expiration / réessai	Top 3 → 10 min → candidat suivant → 5 min → alerte marchand (cron chaque minute)	🟢 8/10
GPS temps réel	Redis Geo + persistance DB + tracé (50 points) + diffusion WebSocket au client	🟢 9/10
Validation livraison	Double étape : livreur marque « livrée » + le client confirme avec un code PIN	🟢 9/10
Paiements	Séquestre, index unique anti-double-paiement, réconciliation périodique, crédit gains livreur automatique	🟢 9/10
Résilience du dispatch	Dépendance forte aux livreurs internes — pas de secours	🟠 5/10
Gestion « aucun livreur »	Point critique : aucune solution de repli	🔴 3/10

3. Architecture cible : dispatch multi-niveaux

Principe fondateur : « le dispatch interne a échoué » n'est pas une erreur finale — c'est un passage automatique au niveau de secours suivant.

Le DispatchService devient un orchestrateur qui escalade à travers 4 niveaux :

Niveau	Moyen de livraison	Rôle
1 — Livreurs internes	Top 3 candidats → retry (existant)	Distribution rapide à faible coût
2 — Agences partenaires	Flottes locales par zone (à construire)	Résout la pénurie de personnel
3 — Providers tiers	GOExpress/WhatsApp (manuel) ou API future	Débordement élargi
4 — Intervention humaine	Marchand / support / dispatcher	Escalade opérationnelle

Séquence SLA proposée (à calibrer avec le terrain) :

Temps	Événement
T+0	Commande prête (READY_FOR_PICKUP) → dispatch interne
T+0 → T+5	Niveau 1 : notification des 3 meilleurs candidats
T+5 → T+10	Niveau 1 (retry) : candidat suivant
T+10	Niveau 2 : agence partenaire
T+15	Niveau 3 : provider secondaire
T+20	Niveau 4 : escalade support / dispatcher

4. Séparation Provider / Driver (règle d'architecture clé)

On ne mélange plus le « moyen de livraison » (provider) et le « livreur » (driver). Le moteur ne demande plus « quel livreur est disponible ? » mais « quel moyen de livraison satisfait cette commande au meilleur coût / SLA ? »

Flux cible :

Commande → DispatchProvider (choisi par l'orchestrateur) → Driver

Champ du Provider	Description
Type	INTERNAL (livreurs internes) · AGENCY (agence) · THIRD_PARTY (société/API)
Drivers	Livraison avec les livreurs associés (flotte interne ou agence)
Zones	Couverture géographique (Ouagadougou, Bobo-Dioulasso…)
Tarifs	Prix par course ou pourcentage de commission
SLA	Temps de prise en charge, fiabilité
Disponibilité	Actif, capacité maximale en vol, heures de service

5. Plan d'implémentation par phases

Phase	Contenu	Effet attendu
0 — Socle	Abstraction Provider + orchestrateur + registry (env DELIVERY_PROVIDERS)	Aucun changement de comportement ; prépare tout le reste
1 — Agences	Entité agence, rôle, dashboard, commission, débordement N1→N2	Résout la pénurie de livreurs (levier principal)
2 — Providers tiers	Bridge « manuel » (WhatsApp/téléphone) + adaptateur API + webhook HMAC	Débordement vers partenaires
3 — Escalade humaine	Écran dispatcher, file SLA dépassée, assignation 1 clic	SLA garanti et visible
4 — Scoring v2	ETA + 6 facteurs (disponibilité, fiabilité, acceptation, qualité, coût)	Meilleure allocation des courses

Principe de sécurité : les données (SLA, webhooks partenaires) jamais exposées en clair dans les logs ; isolation par zone et par agence.

6. Bénéfices attendus

- Plus de commandes livrées : fallback systématique à chaque niveau d'escalade.
- Zéro régression : le moteur actuel est conservé, la nouvelle couche se branche au-dessus.
- FasoFree ne gère plus la masse salariale : les agences s'occupent de leurs chauffeurs.
- SLA tracé et visible (marchand, livreur, support) — livraison garantie dans un délai connu.
- Evolutif : un nouveau partenaire (agence ou société avec API) s'ajoute sans refonte.

7. Décisions à trancher ensemble

Question	Proposition / rappel
Feu vert Phase 0 seule ou Phase 0 + 1 ensemble ?	Phase 0 seule = risque quasi nul ; 0+1 = solution complète
Calibration des timers SLA (T+5/T+10/T+15/T+20)	Valeurs initiales à confirmer avec le terrain
Commission agence	Ex. 10 % ou 15 % des frais de livraison
Zones cibles	Ouagadougou uniquement, ou Bobo-Dioulasso en relais ?
Source de données pour l'ETA (Phase 4)	OSRM / OpenRouteService (gratuit) ou Google (payant)
Contact d'agences / moto-taxis ?	À initier dès que la Phase 1 démarre
Qui gère le bridge « manuel » en production ?	Équipe support / opérations

8. Prochaines étapes

- **Valider la direction et les décisions ouvertes (section 7).**
- Implémenter la Phase 0 (socle orchestrateur + abstraction Provider) — sans aucun changement de comportement actuel.
- Puis Phase 1 : modèle Agences (dashboard, commission, débordement N1→N2).
- Tests + revue de code systématiques avant chaque mise en production (process habituel).

Merci de votre lecture — vos retours et validations sont les bienvenues.